

Cell 5 — Split Data + Encode Labels (Notebook cell index: 10)

Optional fast subsampling, stratified train/validation split, integer labels for sklearn, one-hot labels for Keras.

Granular line-by-line breakdown

Fast mode flags

```
# Set FAST_MODE=True for debugging only, False for final report runs
FAST_MODE = False
FAST_SAMPLES_PER_CLASS = 2500
```

- **FAST_MODE** — Boolean switch: **False** = use full data.
 - **FAST_SAMPLES_PER_CLASS** — Cap per class when fast mode is on.
-

Working copies

```
work_train = train_df.copy()
work_test = test_df.copy()
```

- **.copy()** — Duplicate DataFrames so subsampling (if enabled) does not alter originals **train_df** / **test_df**.
-

Conditional subsampling

```
if FAST_MODE:
```

- **if FAST_MODE:** — Block runs **only** when flag is True.

```
    work_train = work_train.groupby('label', group_keys=False).apply(lambda
```

- **work_train.groupby('label', ...)** — One group per class label.
- **group_keys=False** — Do not add extra index levels from group keys in result (cleaner index).
- **.apply(lambda x: ...)** — For each group **x** (sub-DataFrame), run the lambda.
- **x.sample(min(len(x), FAST_SAMPLES_PER_CLASS), random_state=SEED)** — Take up to **FAST_SAMPLES_PER_CLASS** rows **from that class**; **min** prevents requesting more rows than exist in a small class.
- **random_state=SEED** — Reproducible random subsample.
- **.reset_index(drop=True)** — Flatten row index to 0..N-1 after groupby/apply.

```
    work_test = work_test.groupby('label', group_keys=False).apply(lambda
```

- Same idea for test set with a **minimum** of 700 samples per class when possible (**max(700, FAST_SAMPLES_PER_CLASS // 2)** caps/lifts test size per class).

If FAST_MODE stays False: Entire block skipped—full data used.

Train / validation split

```
train_part, val_part = train_test_split(
    work_train,
    test_size=0.15,
    random_state=SEED,
    stratify=work_train['label']
)
```

- **train_test_split** — Returns **two** subsets: here named **train_part** (85%) and **val_part** (15%).
- **work_train** — DataFrame being split (only training pool; test set **work_test** stays separate).
- **test_size=0.15** — Validation fraction = 15%.
- **random_state=SEED** — Reproducible shuffle/split.
- **stratify=work_train['label']** — Keeps **approximately the same class proportions** in **train_part** and **val_part**. **If omitted:** small classes might be unevenly split → unreliable validation metrics.

Label encoder

```
label_encoder = LabelEncoder()
```

- Constructs an empty encoder object.

```
label_encoder.fit(work_train['label'])
```

- **fit** — Learns the **universe of class names** from **all** labels in **work_train** (full training pool before train/val split). **Why:** Ensures every class that appears in training has an index; consistent mapping when transforming **work_test['label']** too.

```
y_train = label_encoder.transform(train_part['label'])
y_val = label_encoder.transform(val_part['label'])
y_test = label_encoder.transform(work_test['label'])
```

- **transform** — Maps each string label to integer **0 .. num_classes-1** using the **same** mapping for train, val, and test.
- **train_part['label']** — Series of strings aligned with **train_part** rows.

Class count and one-hot encoding

```
num_classes = len(label_encoder.classes_)
```

- **label_encoder.classes_** — NumPy array of class names in encoder order.
- **len(...)** — Number of distinct classes **K**

```
y_train_oh = to_categorical(y_train, num_classes=num_classes)
y_val_oh = to_categorical(y_val, num_classes=num_classes)
y_test_oh = to_categorical(y_test, num_classes=num_classes)
```

- **to_categorical** — Converts integer labels to **one-hot** matrices shape (n_samples, num_classes).
- **num_classes=** — Explicit class count (avoids ambiguous shapes if labels do not include every class in a subset).

If omitted for neural nets: categorical_crossentropy does not match label shape.

Prints

```
print('Train split:', train_part.shape)
print('Val split:', val_part.shape)
print('Test split:', work_test.shape)
print('Classes:', list(label_encoder.classes_))
```

- **.shape** — Rows × columns for sanity check.

- `list(label_encoder.classes_)` — Human-readable ordered list of topic names (matches metric reports).
-

Mechanism of the whole cell

Copy data → optionally subsample → split **training pool** into train/val → fit label mapping on training labels → integer `y_*` for sklearn → one-hot `y_*_oh` for TensorFlow.

Role and importance in the notebook

Defines **which rows tune hyperparameters** (val) vs **fit models** (train) vs **final evaluation** (test), and ensures **consistent label IDs** everywhere.